

Handlebars template injection and RCE in a Shopify app

Handlebars template injection and RCE in a Shopify app - Mahmoud Gamal, Blogger.

- Published: 2019-04-04
- Original: <<https://mahmoudsec.blogspot.com/2019/04/handlebars-template-injection-and-rce.html>>
- Preserved from: <https://mahmoudsec.blogspot.com/2019/04/handlebars-template-injection-and-rce.html> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Handlebars template injection and RCE in a Shopify app

TL;DR

We found a zero-day within a JavaScript template library called handlebars and used it to get Remote Code Execution in the Shopify Return Magic app.

The Story:

In October 2018, Shopify organized the HackerOne event "H1-514" to which some specific researchers were invited and I was one of them. Some of the Shopify apps that were in scope included an application called "Return Magic" that would automate the whole return process when a customer wants to return a product that they already purchased through a Shopify store.

Looking at the application, I found that it has a feature called Email WorkFlow where shop owners can customize the email message sent to users once they return a product. Users could use variables in their template such as `{{order.number}}` , `{{email}}` ..etc. I decided to test this feature for Server Side Template injection and entered `{{this}}` `{{self}}` then sent a test email to myself and the email had `[object Object]` within it which immediately attracted my attention.

So I spent a lot of time trying to find out what the template engine was, I searched for popular NodeJs templates and thought the template engine was mustache (wrong), I kept looking for mustache template injection online but nothing came up as Mustache is supposed to be a logicless template engine with no ability to call functions which made no sense as I was able to call some Object attributes such as `{{this.proto}}` and even call functions such as `{{this.constructor.constructor}}` which is the Function constructor. I kept trying to send parameters to `this.constructor.constructor()` but failed.

I decided that this was not vulnerable and moved on to look for more bugs. Then the fate decides that this bug needs to be found and I see a message from Shopify on the event slack channel asking researchers to submit their "almost bugs" so if someone found something and feels it's exploitable, they would send the bug to Shopify security team and if the team manages to exploit it the reporter will get paid as if they found it. Immediately I sent my submission explaining what I have found and at the impact section I wrote "Could be a Server Side template injection that can be used to take over the server ``\(\)/``".

Two months passed and I got no response from Shopify regarding my "almost bug" submission, then I was invited to another hacking event in Bali hosted by Synack. There I met the Synack Red Team and after the Synack event has ended, I was supposed to travel back to Egypt, but only 3 hours before the flight I decided to extend my stay for three more days then fly from Bali to Japan where I was supposed to participate in the TrendMicro CTF competition with my CTF team. Some of the SRT also decided to extend their stay in Bali. One of those was Matias so I contacted him to hangout together. After swimming in the ocean and enjoying the beautiful nature of Bali, we went to a restaurant for dinner where Matias told me about a bug he found in a bug bounty program that had something to do with JavaScript sandbox escape so we spent all night missing with objects and constructors, but unfortunately we couldn't escape the sandbox.

I couldn't take constructors out of my head and I remembered the template injection bug I found in Shopify. I looked at the HackerOne report and thought that the template can't be mustache so I installed mustache locally and when I parsed `{{this}}` with mustache it actually returns nothing which is not the case with the Shopify application. I searched again for popular NodeJs template engines and I found a bunch of them, I looked for those that used curly brackets `{{ }}` for template expressions and downloaded them locally, one of the libraries was handlebars and when I parsed `{{this}}` it returned `[object Object]` which is the same as the Shopify app. I looked at handlebars documentation and found out that it's also supposed to not have much logic to prevent template injection attacks. But knowing that I can access the function constructor I decided to give it a try and see how I can pass parameters to functions.

After reading the documentation, I found out that in handlebars developers can register functions as helpers in the template scope. We can pass parameters to helpers like this `{{helper "param1" "param2" ...params}}`. So the first thing I tried was `{{this.constructor.constructor "console.log(process.pid)"}}` but it just returned `console.log(process.pid)` as a string. I went to the source code to find out what was happening. At the runtime.js file, there was the following function:

So what this function does is that it checks if the current object is of type 'function' and if so it just calls it using `current.call(context)` where context is the template scope, otherwise, it would just return the object itself.

I looked further in the documentation of handlebars and found out that it had built in helpers such as "with", "blockHelperMissing", "forEach" ...etc

After reading the source code for each helper, I had an exploitation in mind using the "with" helper as it is used to shift the context for a section of a template by using the built-in with block helper. So I would be able to perform `current.call(context)` on my own context. So I tried the following:

Basically that should pass `console.log(process.pid)` as the current context, then when the handlebars compiler reaches `this.constructor.constructor` and finds that it's a function, it should call it with the current context as the function argument. Then using `{{#with this}}` we call the returned function from the Function constructor and `console.log(process.pid)` gets executed.

However, this did not work because `function.call()` is used to invoke a method with an owner object as an argument, so the first argument is the owner object and other arguments are the parameters sent to the function being called. So if the function was called like `current.call(this, context)`, the previous payload would have worked.

I spent two more nights in Ubud then flew to Tokyo for the TrendMicro CTF. Again in Tokyo, I couldn't take objects and constructors out of my mind and kept trying to find a way to escape the sandbox.

I had another idea of using `Array.map()` to call Function constructor on my context, but it didn't work because the compiler always passes an extra argument to any function I call which is an object containing the template scope which causes an error as my payload is considered a function argument not the function body.

[[image: image]]

(https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvXsEiMujYnFAeaBgUip4mEusio-ur-EOpn-gQQ6cxhETQ3S_xw4eGYRPeLko9nAlMv2eWXhwGl-ajk6GDRjCYLOB-zozeApPFBbuYa2jeA8gVEgTcci4hAS3Jec3P0ZX4VcbpqUVjJhcsY2vhu/s1600/Screen+Shot+2019-04-01+at+10.50.40+PM.jpg)

There seemed to be many possible ways to escape the sandbox but I had one big problem facing me which is that whenever a function is called within the template, the template compiler sends the template scope Object as the last parameter.

For example, if I try to call something like `constructor.constructor("test","test")`, the compiler will call it like `constructor.constructor("test", "test", this)` and this will be converted to a string by calling `Object.toString()` and the anonymous function created will be: which will cause an error.

I tried many other things but still no luck, then I decided to open the JavaScript documentation for Object prototype and look for something that could help escape the sandbox.

I found out that I could overwrite the `Object.prototype.toString()` function using `Object.prototype.defineProperty()` so that it calls a function that returns a user controlled string (my payload).

Since I can't define functions using the template, all I have to do is to find a function that is already defined within the template scope and returns a user controlled input.

For example, the following nodejs application should be vulnerable: test.js example.html

Now if you run this template, `console.log(process.pid)` gets executed. I reported that to Shopify and mentioned that if there was a function within the scope that returns a user controlled string, it would have been possible to get RCE.

Later, when I met Ibrahim (@the_st0rm) I told him about my idea and he told me that I can use `bind()` to create a new function that when called will return my RCE payload. From JavaScript documentation:

The `bind()` method creates a new function that, when called, has its `this` keyword set to the provided value, with a given sequence of arguments preceding any provided when the new function is called.

So now the idea is to create a string with whichever code I want to execute then bind its `toString()` to a function using `bind()` after that overwrite the `Object.prototype.toString()` function with that function.

I spent a lot of time trying to apply this using handlebars templates, and eventually during my flight back to Egypt I was able to get a fully working PoC with no need to use functions defined in the template scope.

[[image: image]]

(https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvXsEhZgfGjx4gINW6O_cVPLw82glZFy9Zpw146TIRu0OfBiE_W6MqrNAe4CHFERb3rHGccPlc-l0KpQia1Lgzg8Gu6hj-BX2ZXmJt6IPdigexxd4QShVzq1BUgxCS5bgbV1wT84W0h8PP2QESN7/s1600/Screen+Shot+2019-04-02+at+12.38.24+AM.jpg)

Basically, what the template above does is:

And when I tried it with Shopify, I got:

[[image: image]]

(https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvXsEjHF99lpwEXrNTpBOocrgPKwkq0EV86nMRw55ca0zytoqC1he-dTPmF5pEyRNqNVG-8WSnefCnenAiUW4dYb99cG5OGOr2LpSVVTBLzFCr6kx51B5CLGTjY90GIxgPGsPpVbWJlpRiubnub/s1600/Screen_Shot_2018-12-16_at_6.56.00_PM.jpg)

Matias also texted me with an exploitation that he got which is much simpler than the one I used:

With that said, I was able to get RCE on Shopify's Return Magic application as well as some other websites that used handlebars as a template engine.

The vulnerability was also submitted to npm security and handlebars pushed a fix that disables access to constructors. The advisory can be found here: <https://www.npmjs.com/advisories/755>

In a nutshell

You can use the following to inject Handlebars templates:

Matias also had his own exploitation that is much simpler:

Sorry for the long post, if you have any questions please drop me a tweet [@Zombiehelp54](#)

Comments

Load more...

Archived from <https://mahmoudsec.blogspot.com/2019/04/handlebars-template-injection-and-rce.html>.

Rendered offline from the local Markdown copy.